

情報ネットワーク学演習Ⅰ 最終レポート

：内田 岳 33E25020：田中 晴輝

：竹内 章一郎：宮田 成人

提出日：2025 年 10 月 29 日

1 アイデアとアプリケーション

本節では，本グループのアイデアおよび実装したアプリケーションについて説明する．

ドローンは，災害現場での搜索やインフラ設備の点検，空撮，ドローンショーといった多様な分野で活用されており，需要が高まっている．しかしながら，日常生活の中でドローンに触れる機会は依然として限られており，高齢者や技術の関心が薄い層には普及していない．また，幅広い世代がドローンを身近に感じるには，遊びや体験を通じた企画が重要となる．そこで本グループでは，図 6b のようにデパートの屋上などに設置され，世



(a) ドローンショー



(b) デパートの屋上のモグラたたき

図 1: (a) ドローンの活用例 [1], (b) モグラたたきの例 [2]

代を問わず親しまれてきた「モグラたたき」に注目し、ドローンと組み合わせたアプリケーションを実装した。これにより、ドローンを楽しみながら体験する機会を提供し、多くの人に身近に感じてもらうことを目指す。具体的には、Raspberry Pi を用いてドローンを操作して、モグラの役割である複数の Raspberry Pi にドローンを近づけて叩く。

2 システム

本節では、「ドローンでモグラたたき」の全体像について説明する。

図 2 にシステムの全体像を示す。本システムは、Raspberry Pi (コントローラー)、Raspberry Pi 4 台 (モグラ)、Raspberry Pi (集中制御)、Crazyflie 2.1 (ドローン)、ノート PC 2 台、得点ランキング表示アプリケーションから構成される。Raspberry Pi (コントローラー、モグラ) には SenseHat を取り付けた。コントローラーでは、ユーザーが動かしした情報を IMU を用いて読み取り、ドローンの移動の指示を作成してノート PC に伝送する。ノート PC では、受った指示を用いてドローンを動かしモグラを叩きにいく。集中制御では、タイムアウトまでの時間、得点、出現するモグラを管理し、一定間隔でモグラに指示を送る。さらに、ゲーム終了時には得点をノート PC に伝送し、得点ランキング表示アプリケーションに反映させて得点を表示する。モグラでは、集中制御や他のモグラの指示を用いて出現するかしないか決定する。また、磁気センサを用いて磁場の変化を検出し、叩かれたか判定を行う。

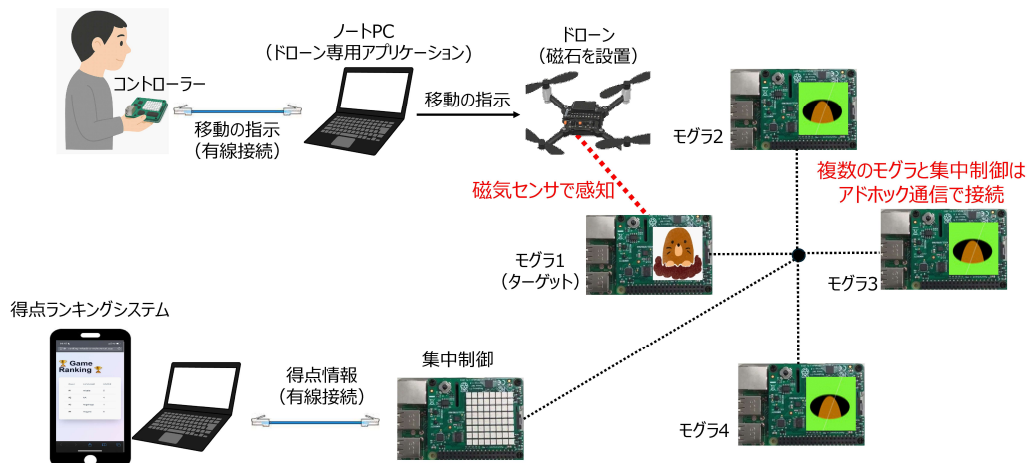


図 2: システムの全体像

3 プログラム

本節では、システムの各構成要素の詳細および実装したプログラムを説明する。

3.1 コントローラの処理

コントローラー用の Raspberry Pi に SenseHat を取り付け、Pitch, Roll, Yaw を API から取得する。リアルタイムに取得しながら、あらかじめ決めた閾値を超えたかどうかと＋方向と－方向どちらで超えたかで”-1”, ”0”, ”1”を決め、ドローン制御用 PC に送信する。データは Pitch, Roll, Yaw の取得ごとにそれぞれに対応する 3 個分の値を送る。送信するデータは 3 つのうち 2 つは必ず”0”になるようにするような処理を行う。具体的には、Pitch, Roll, Yaw のうち 2 つ以上が閾値を超えた場合は、閾値を超えた値が一番大きい値に対応する値以外は”0”にする。また、Pitch, Roll, Yaw に対してオフセット処理を用いたキャリブレーションを行っている。さらに、キャリブレーションをプログラムの開始時のみしか行わない場合は、ドリフトにより誤差が大きくなっていくため、バックグラウンドで定期的にオフセットの更新を行っている。定期的なオフセットの更新を Pitch, Roll, Yaw に対応する値が”0”に近くない時に行ってしまうと操作が困難になるため、全ての値が”0”つまり閾値以下の時にのみオフセットの更新を行うようにしている。

3.2 ドローンの飛行操作

3.2.1 実装環境

ドローンは、Bitcraze 社の Crazyflie 2.1 という超小型クワッドコプタを用いた [3]。ドローン本体の重量は 27 g, サイズは 92 mm × 92 mm × 29 mm で、2.4 GHz 帯を用いたマルチプラットフォームな遠隔手動操縦及びプログラムによる遠隔操縦が可能である。3 軸加速度センサー、ジャイロスコープ (BMI088) 及び高精度圧力センサー (BMP388) を搭載している。測位システムとして、SteamVR Base Station 2.0[4] と Lighthouse positioning deck[5] を使用した赤外線の到来角推定を用いた。SteamVR Base Station 2.0 の位置はキャリブレーションが必要であり、飛行前にこれを行った後に、コントローラーの Raspberry Pi と有線接続した PC を用いて、ドローンの飛行操作を行った。

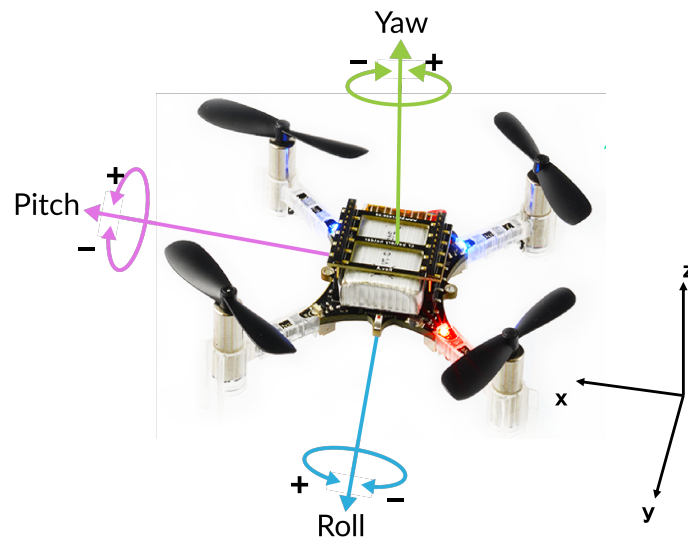


図 3: ドローンの軸の対応

3.2.2 実装詳細

まずドローンをホバリングさせる指令を出した後，コントローラー用の Raspberry Pi との UDP を用いた Socket 通信を待機状態にする．次に 3.1 で説明した Pitch, Roll, Yaw をもとに飛行の指示を与える． (x,y,z) の対応は図 3 のとおりであり，これをもとに以下のような飛行指令をドローンに送信する．

- Roll==1 のとき，x 軸の正の方向に一定速度で進む
- Roll==−1 のとき，x 軸の負の方向に一定速度で進む
- Pitch==1 のとき，y 軸の正の方向に一定速度で進む
- Pitch==−1 のとき，y 軸の負の方向に一定速度で進む
- Yaw==1 又は Yaw==−1 のとき，z 軸の負の方向に 10 cm 進んだ後に，z 軸の正の方向に 10 cm 進む

3.3 集中制御およびモグラの動作

表 1 に集中制御およびモグラにおける指示と動作を示す．

表 1: 集中制御およびモグラにおける指示と動作

指示	動作
“0”	穴に戻れ！
“1”	モグラが出現しろ！
“2”	モグラ出現しました(タイムアウト時間計測用)
“3”	モグラが叩かれました(得点管理用)

3.3.1 集中制御

集中制御は主に4つのスレッドで実装した。1つ目のスレッドでタイムアウトまでの時間を監視する。2つ目のスレッドでゲームのプレイ終了までの時間を監視する。3つ目のスレッドでモグラからの受信を待機する。4つ目のスレッドでモグラから受信したデータに合わせてタイムアウトの基準時間の更新であったり、どのモグラが叩かれたのかというのをカウントする。また、ゲーム開始時とタイムアウト時に関しては、集中制御から4つのモグラに対して1つだけをランダムに選択して”1”を送信し、それ以外には”0”を送信する。さらに、ゲームのプレイ時間が終わった場合には、データベース更新用プログラムを実行しているpcに対して、叩かれた合計回数を送信して集中制御のプログラムは終了する。

3.3.2 モグラの動作

モグラはソケット通信を利用して、他のモグラや集中制御の指示を常に受け取れるようにする。また、モグラが出現しているかは変数「led_on」で管理する。ソケット通信で”1”を受信した場合は、SenseHat をモグラ色に光らせ、集中制御に”2”を伝送する。”0”を受信した場合は、SenseHat の光り方を変更する。これらは、以下のように実装した。

```

1 def receive_message():
2     global led_on
3
4     server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5     server_sock.bind((my_ip, PORT))
6     server_sock.listen()
7
8     while True:
9         try:
10             conn, addr = server_sock.accept()

```

```

11         with conn:
12             msg = conn.recv(1024).decode().strip()
13             print(f"Received-msg:{msg}")
14
15             if msg == "1":
16                 sense.set_pixels(mole)
17                 led_on = True
18                 send_to_central("2")
19
20             elif msg == "0":
21                 #草むら色に変更
22                 led_on = False
23
24     except Exception as e:
25         print("error")
26         continue

```

また、モグラは磁場の変化を感知するプログラムをスレッドで並行実行し、常に磁場の変化を監視する。モグラとして出現している状態で、磁場の変化が設定した閾値を超えた場合には、叩かれたと判定し、集中制御に”3”を送信する。加えて、他のモグラのランダムに選んだ1台に”1”を、残りに”0”を送信する。叩かれた際にモグラ間でアドホック通信を活用して直接指示を送信することで、集中制御を介した場合より低遅延（ワンホップ）となる。実装したプログラムは以下であり、磁気感知システムの詳細は次節で示す。

```

1     def monitor_magnetic_change():
2         baseline = initialize_baseline()
3         prev_total_mag = baseline
4
5         while True:
6             triggered, new_total_mag = detect_magnetic_change(prev_total_mag,
7                 , baseline, delta_threshold=0.6)
8             prev_total_mag = new_total_mag
9
10            if triggered:
11
12                if not led_on:
13                    print("出現してないよ")
14                    continue
15
16                print("叩かれた!")
17                send_to_central("3")
18                send_random_to_others()
19
20            prev_total_mag = wait_until_stable()
21            baseline = prev_total_mag

```

```
21
22         time.sleep(0.3)
```

3.3.3 磁気感知システム

磁気感知システムでは下記のプログラムをスレッドで並行実行している. (3.3.2 節で記載した `monitor __ magnetic __ change()` のフルバージョン) このプログラムには主に `baseline` の初期化, 磁場変化監視ループ, 誤検知対策の 3 つの動作がある.

```
1 def monitor_magnetic_change():
2     baseline = initialize_baseline()
3     prev_total_mag = baseline
4
5     while True:
6         triggered, new_total_mag = detect_magnetic_change(prev_total_mag,
7                                                             baseline, delta_threshold=0.12)
8         prev_total_mag = new_total_mag
9
10        if triggered:
11
12            if not led_on:
13                print("出現してないよ")
14                prev_total_mag = wait_until_stable()
15                baseline = prev_total_mag
16                continue
17
18            sense.clear()
19            sense.set_pixels(dead_mole)
20            time.sleep(0.3)
21            sense.clear()
22            sense.set_pixels(back)
23
24            print("叩かれた!")
25            send_to_central("3")
26            send_random_to_others()
27
28            time.sleep(0.1)
```

`baseline` の初期化は下記のプログラムで行われている. 起動時は磁場が大きく変動するので, 磁場センサーからの出力を一定間隔で取得し, 磁場の大きさが安定した状態を検出して, そのときの値を `baseline` として記録する. 具体的には, 磁場の変化量が指定したしきい値 `stable __ threshold` 未満の状態が, 連続して `stable __ count __ required` 回続いたときに磁場が安定していると判定する. 最後に記録した磁場の大きさを基準値として

返す.

```
1 def initialize_baseline(stable_threshold=0.5, stable_count_required=10,
2   interval=0.1):
3     print("初期化中磁場が安定するのを待っています...")
4     stable_count = 0
5
6     mag = sense.get_compass_raw()
7     x, y, z = mag['x'], mag['y'], mag['z']
8     prev_total_mag = (x**2 + y**2 + z**2) ** 0.5
9
10    while True:
11        time.sleep(interval)
12        mag = sense.get_compass_raw()
13        x, y, z = mag['x'], mag['y'], mag['z']
14        total_mag = (x**2 + y**2 + z**2) ** 0.5
15        delta = abs(total_mag - prev_total_mag)
16
17        if delta < stable_threshold:
18            stable_count += 1
19        else:
20            stable_count = 0
21
22        if stable_count >= stable_count_required:
23            time.sleep(0.6)
24            break
25
26        prev_total_mag = total_mag
27
28    baseline = total_mag
29    print("初期化完了ベースライン:=", baseline)
30    return baseline
```

磁場変化監視ループは、`monitor __ magnetic __ change()` 内のループで下記のプログラムを実行することで実装されている。現在の磁場の大きさを取得し、直前の値との変化量を計算して、急激な変化があったかどうかを判定する。変化量が指定しきい値 (0.12) を超え、かつ現在の磁場がベースラインより 0.6 以上大きい場合に「叩かれた」と判定する。

```
1 def detect_magnetic_change(prev_total_mag, baseline, delta_threshold):
2     mag = sense.get_compass_raw()
3     x, y, z = mag['x'], mag['y'], mag['z']
4     total_mag = (x**2 + y**2 + z**2) ** 0.5
5     delta = total_mag - prev_total_mag
6     return (delta > delta_threshold) and (total_mag > baseline + 0.6),
7         total_mag
```


誤検知対策は、`monitor __ magnetic __ change()` で、モグラが出現していないにもかかわらず磁場の大きな変化を感知し、「叩かれた」と誤判定してしまう場合に行われる。磁場は環境要因によって常に変動しており、しきい値を一時的に超えることもあるため、こうした誤検知が発生する可能性がある。このような誤検知が発生した場合には、下記の `wait __ until __ stable()` によって磁場の安定状態を再確認し、`baseline` を再設定する処理が実行される。

```
1 def wait_until_stable(stable_threshold=0.5, stable_count_required=10,
2   interval=0.1):
3     print("磁場が安定するのを待っています...")
4     stable_count = 0
5     mag = sense.get_compass_raw()
6     x, y, z = mag['x'], mag['y'], mag['z']
7     prev_total_mag = (x**2 + y**2 + z**2) ** 0.5
8
9     while True:
10        time.sleep(interval)
11        mag = sense.get_compass_raw()
12        x, y, z = mag['x'], mag['y'], mag['z']
13        total_mag = (x**2 + y**2 + z**2) ** 0.5
14        delta = abs(total_mag - prev_total_mag)
15
16        print("安定化チェック:", total_mag, "変化:", delta)
17
18        if delta < stable_threshold:
19            stable_count += 1
20        else:
21            stable_count = 0
22
23        if stable_count >= stable_count_required:
24            print("磁場が安定しました")
25            print("ベースライン␣=", total_mag)
26            time.sleep(0.6)
27            sense.clear()
28            return total_mag
29
30    prev_total_mag = total_mag
```

3.4 ランキング表示アプリケーション

図4はランキング表示アプリケーションのシステム構成である。

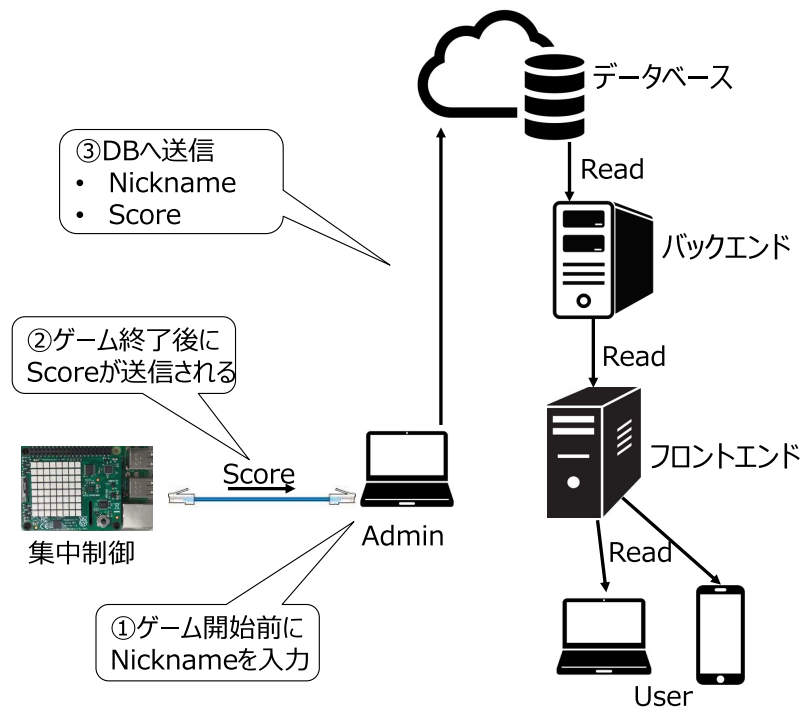


図 4: ランキング表示アプリケーションのシステム構成

3.4.1 ランキング表示アプリケーションの実装

本アプリケーションは、ゲームのランキング表示システムであり、ユーザーの Nickname と最高得点を記録・表示することを目的としている。実装のソースコードは、<https://github.com/tanakaharuki0/ranking-whack-a-mole> に公開している。データベース、バックエンド、フロントエンドの三層アーキテクチャに加え、Nickname と Score を送信する Admin 用の PC で構成されている。無料で使用できるサーバとして、データベースとバックエンドは Render を、フロントエンドは vercel を使用し、それぞれで必要な環境変数を設定して動作させた。

データベース: 本アプリケーションでは PostgreSQL を採用した。データベースでは、scores テーブルを定義し、以下のデータを保持する。

- id: SERIAL PRIMARY KEY. 各エントリを一意に識別する自動増分整数。
- nickname: TEXT NOT NULL. ユーザーの Nickname を文字列として格納。NULL を許容しない。
- score: INTEGER NOT NULL. Score を整数として格納。NULL を許容しない。

- timestamp: `TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP`.
Score が記録された日時をタイムゾーン情報付きで自動記録.

バックエンド: バックエンドは Python の Flask フレームワークを使用し, データベースに接続するためのライブラリとして `psycopg2` を用いた. データベースの接続情報は環境変数 `DATABASE_URL` から取得される. 実装したエンドポイントは以下のとおりである.

1. **GET /api/scores:**

- 目的: 最新のゲームランキング (各 Nickname の最高スコア上位 10 件) を取得する.
- 処理: データベースから SQL クエリを実行し, 結果を JSON 形式で返す. エラー発生時には 500 エラーを返す.

2. **POST /api/score:**

- 目的: 新しいスコアをデータベースに保存する.
- 処理: リクエストボディから `nickname` と `score` を JSON 形式で受け取る. データの型検証を行い, 不正な形式の場合は 400 エラーを返す. 問題なければデータベースにスコアを挿入し, 201 Created ステータスを返す.

3. **DELETE /api/score/{nickname}:**

- 目的: 指定された Nickname に関連するすべてのスコアをデータベースから削除する.
- 処理: URL パスから `nickname` を取得し, その Nickname のすべてのエントリを削除する. 削除された場合は 200 OK, スコアが見つからなかった場合は 404 Not Found を返す.

4. **OPTIONS /api/score/{nickname}:**

- 目的: CORS のプリフライトリクエストに対応する.
- 処理: `Access-Control-Allow-Methods` と `Access-Control-Allow-Headers` ヘッダーを設定し, 許可される HTTP メソッドとヘッダーをクライアントに通知する.

フロントエンド: フロントエンドは Next.js フレームワークを用いて構築し, クライアントサイドレンダリング方式を採用した. バックエンドをデプロイしているサーバの URL は環境変数 `NEXT_PUBLIC_APIBASE_URL` から取得される. 表示内容と機能は以下のとおりである.

- サーバーから取得したスコアデータ `scores` を利用し、テーブル形式でランキングを表示する.
- 各行には「Rank」「Nickname」「Score」が表示される.
- スコアがない場合は、「No scores yet. Play a game!」というメッセージが表示される.

Admin 用 PC: Admin 用 PC では、ソケット通信を通じて外部から Score データを受信し、その Score と Nickname を直接 PostgreSQL データベースに保存するという設計である. 環境変数は `".env.local"` というファイル内にローカルで保持しているため、`".gitignore"` を用いて非公開にしている.

3.4.2 ランキング表示アプリケーションの使用方法

ランキング表示アプリケーションの使用手順は以下のとおりである.

1. Nickname を入力し、ゲームを開始する
2. ゲーム終了後に集中制御の Raspberry Pi から Score が送信される
3. Nickname と Score が自動でデータベースに送信される
4. ランキング表示アプリケーションにアクセスし、図 5 のようなランキングを確認する

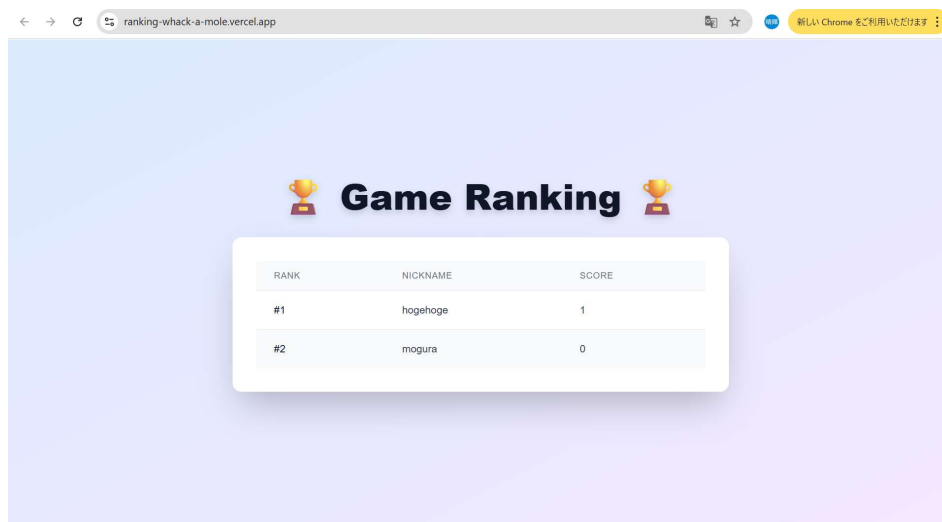
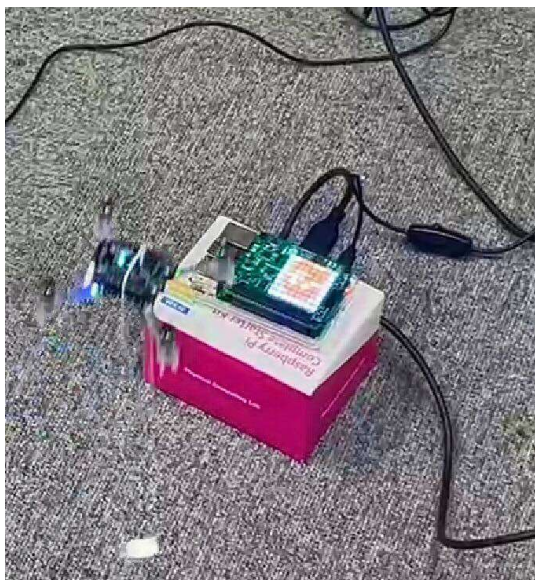


図 5: ランキング表示アプリケーションの UI

4 動作検証

動作検証を行うためのセットアップの手順を以下に示す.

1. 機器のキャリブレーションをし, 測位システムを使用できるようにする
2. ドローンに磁石を搭載する
3. コントローラー用 Raspberry Pi とドローン制御用 PC を有線接続する
4. 4 台のモグラ用 Raspberry Pi を待機状態にする
5. 集中制御の Raspberry Pi とランキング表示機能の Admin 用 PC を有線で繋ぎ, ゲーム参加者の Nickname を入力する



(a) 叩かれる前のモグラ



(b) 叩かれた後のモグラ

図 6: モグラ叩きの検証

続いてゲーム開始から終了までの流れを以下に示す.

1. ドローン制御用 PC でドローンを高さ 35 cm のところでホバリングさせた後に, コ

- ントローラー用 Raspberry Pi を起動して操縦できるようにする
2. ゲーム開始の合図とともに、集中制御の Raspberry Pi を起動してモグラ用 Raspberry Pi の制御を行う
 3. SenseHat にモグラが現れたら、コントローラーでドローンを操縦して磁石を用いて叩く
 4. 磁力の変化量が閾値を超えるとモグラが叩かれた判定となり、モグラが図 6 のようにダメージを受けて SenseHat の UI が変化する
 5. モグラは叩かれるかタイムアウトすると新たなモグラが出現するのでゲーム終了まで叩き続ける
 6. 一定時間でゲームを終了し、叩けたモグラの数がランキング表示機能の Admin 用 PC を経由してデータベースに保存される
 7. <https://ranking-whack-a-mole.vercel.app/> にアクセスしてランキングを確認する（8/24 にデータベースが期限切れするためそれ以降アクセスするには再デプロイが必要）

5 グループ内での分担・役割

内田 岳

- コントローラーの実装
- 集中制御の実装

田中 晴輝

- 得点ランキング表示アプリケーションの実装
- ドローンの飛行調整

竹内 章一郎

- 磁気感知システムの構築
- モグラの実装

宮田 成人

- システムのネットワーク構築
- モグラの実装

6 感想

- スレッドを分けてコードを書くのになれてなく苦労した
- アドホック通信で MAC アドレスが特定できないエラーが解消できず大変だった
- ドローン操縦のリアルタイム性と直感的な操縦を実現するのが大変だった
- 測位システムが vision ベースではなく古典的な手法のみであったため、センサーの不具合による精度の不安定さに苦戦した
- Raspberry Pi コントローラーでドローンを操縦し、制御されたモグラを叩いて、ゲーム終了後に得点のランキング表示するという一連の流れを動作確認できた時は達成感があり嬉しかった
- 磁場の変化は環境要因に大きく左右されるため、適切なしきい値の設定が難しく、何度も調整が必要だった。安定性と感度のバランスを取ることに苦労した。

参考文献

- [1] D. S. JAPAN, “くるぞ、万博。1 year to go. スペシャルドローンショー（大阪／500機）,” <https://droneshow.co.jp/casestudy/6324/>, accessed: 2025-08-03.
- [2] みみこ, “大阪最後の屋上遊園地？松坂屋高槻店で遊んで来ました。” <https://ikimasyoka.hatenablog.com/entry/2018/11/29/222417>, accessed: 2025-08-03.
- [3] Bitcraze, “Crazyflie2.1,” <https://www.bitcraze.io/products/old-products/crazyflie-2-1/>, accessed: 2025-08-03.
- [4] HTC, “Steamvr base station 2.0,” <https://www.vive.com/jp/accessory/base-station2/>, accessed: 2025-08-03.
- [5] Bitcraze, “Lighthouse positioning deck,” <https://www.bitcraze.io/products/lighthouse-positioning-deck/>, accessed: 2025-08-03.